Shokunin

Persistent AI Memory for Developers

Enterprise White Paper v4.2.2
Generated May 2026

62 skills. 9 MCP memory tools. Multi-strategy recall (vector + BM25 + temporal + RRF). Freshness decay with 30-day half-life. Claim verification for stale memory paths. Zero servers, zero API costs, fully offline. MIT licensed.

Shokunin Ecosystem Overview

Shokunin is a comprehensive AI coding ecosystem that gives developers persistent memory across sessions, 62 domain-expert skills that auto-activate, and a fully offline architecture. The memory system uses three complementary retrieval strategies -- vector similarity (semantic), BM25 (keyword), and temporal (recency) -- fused via Reciprocal Rank Fusion to ensure the most relevant context surfaces for every query.

Skills cover infrastructure (Docker, Kubernetes, Terraform), backend (auth, APIs, databases), frontend (React/Vue/Svelte with Emil Kowalski and Paul Bakaus design standards), mobile (Flutter and React Native), quality (testing, performance, code review), and content/business domains.

Memory Architecture

The memory system is built on ChromaDB with a custom MCP (Model Context Protocol) server providing 9 tools for context storage, retrieval, and management. Data lives entirely on the developer's machine at ~/.shokunin/memory/ -- no cloud, no telemetry, no subscriptions.

Core storage layers:

- ChromaDB vector store for semantic similarity search
- BM25 full-text index for keyword recall
- JSONL session transcripts for replay and debugging

- Markdown fallback files for human-readable history
- Freshness decay blending: exponential recency with 30-day half-life

Search methods:

- search_context -- pure vector similarity with optional type/tag/project filters
- multi_search_context -- vector + BM25 + temporal, fused via RRF
- Both methods accept freshness_boost (0.0--1.0) to blend relevance with recency

Session Management

Shokunin introduces explicit session management. Every AI interaction belongs to a session identified by session-YYYYMMDD-HHMMSS-NNNN. The agent lists recent sessions at startup, lets the user choose which one to continue, and loads full context from that session including all decisions, files, commands, and preferences.

- No guessing -- user explicitly selects which session to resume
- Full context load on session continue (not just a summary)
- Consolidation support for summarizing old entries per project

MCP Tool Suite -- 9 Tools

The memory MCP server exposes 9 tools (up from 8 in v4.2.1), each with enriched schemas including freshness_boost parameters for time-weighted retrieval:

- store_context -- Store entries with type classification and tags
- search_context -- Vector similarity search with freshness blending
- multi_search_context -- Vector + BM25 + RRF + temporal filtering
- get_session_summary -- Aggregate overview of a session
- continue_session -- Load full context from a session
- list_sessions -- Browse recent sessions with metadata
- consolidate_memories -- Summarize old entries per project
- save_message -- Record individual message exchanges
- verify_file_path -- Validate file/directory existence on local filesystem

Multi-Runtime Compatibility

Shokunin works across multiple AI coding environments. Skills are runtime-agnostic SKILL.md files with trigger-optimized descriptions. Memory integrates via MCP or rule files. Config templates are provided for OpenCode (native), Claude Code, Cline, Cursor, Continue.dev, and Windsurf.

Production Quality

Every component is validated in CI on every push. Skills are checked for frontmatter, workflow completeness, error handling patterns, and cited sources. The memory system has 29 integration tests covering security, MCP protocol compliance, and ChromaDB operations. 12 PowerShell scripts use Set-StrictMode, ErrorActionPreference, and CmdletBinding throughout.

- GitHub Actions CI with Windows + Linux matrix
- Memory test suite with one-command validation (test-memory.ps1 / memory-healthcheck.ps1)
- OWASP-informed security patterns across all web features
- Benchmark suite for recall performance and RRF scoring

Retrieval Shaping -- Memory That Respects Time

The hardest problem in persistent memory is not storage -- it is retrieval. Shokunin solves this with exponential freshness decay blended with vector similarity. Each memory entry decays over a 30-day half-life, so recent context surfaces before stale claims. The agent can control the blend: freshness_boost=0.0 for pure relevance, 1.0 for pure recency, or any value between.

When an agent recalls 'the auth helper lives at src/x.ts,' it does not act blindly. The new verify_file_path MCP tool validates file paths before the agent proceeds. If the file no longer exists, the agent searches memory for newer entries describing the refactoring, then greps the actual codebase. Memory is treated as a claim from a frozen point in time -- never as fact.

What Is New in v4.2.2

- Freshness decay: exponential recency blending with 30-day half-life across all search methods
- Claim types: claim_file, claim_function, claim_flag, claim_api for structured fact tracking
- verify_file_path MCP tool: validates file/directory existence before agents act on stale claims
- 9 MCP tools (up from 8): all with freshness_boost parameter and enriched schemas
- Claim Verification Rule in agent instructions: memory paths must be verified before use
- 30+ bug fixes: recursion, filter overwrite, RRF session collision, BM25 duplication, similarity formulas

- 29 integration tests covering security, MCP protocol, and ChromaDB operations
- 12 PowerShell scripts with Set-StrictMode, ErrorActionPreference, CmdletBinding

Getting Started

Windows (PowerShell):

```
irm https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.ps1 | iex
```

Linux (bash):

```
bash <(curl -sL https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.sh)
```

Requirements: Python 3.11+, Node.js 18+, Git. The installer sets up everything: OpenCode CLI, ChromaDB, PowerShell/Bash profiles, MCP server configuration, and skill files. Start a session with opencode (or .\run-opencode.ps1 for memory capture on Windows).

License & Links

- MIT License -- free as in freedom, free as in zero cost
- GitHub: github.com/EliasOulkadi/shokunin
- Website: eliasoulkadi.github.io/shokunin
- Documentation: docs/ARCHITECTURE.md, CHANGELOG.md

Shokunin v4.2.2 -- Persistent memory, multi-strategy recall, and claim verification for developers who demand production-grade tooling. Built for Windows 11 and Linux. Open source since 2024.